Resultados Preliminares: Modelos de Concorrência num Microserviço Java

[Nome do autor]
Programação Paralela e Concorrente
Mestrado em [nome do curso]
Faculdade de Ciências, Universidade de Lisboa
Lisboa, Portugal
[email institucional]

Abstract

Este trabalho compara três modelos de concorrência num microserviço Java: platform threads, virtual threads e I/O assíncrono. Foi implementado um protótipo com um serviço HTTP simulado e executado um piloto com k6, a 10, 100 e 500 utilizadores virtuais, usando um atraso de 1.000 ms no serviço simulado. Nesta primeira execução, os três modelos apresentaram resultados semelhantes; são dados exploratórios e não permitem ainda conclusões gerais.

1 Objetivo e trabalho realizado

A pergunta de investigação é: como varia o throughput e a latência de um microserviço quando aumenta a concorrência, comparando diferentes modelos de execução? Foi desenvolvido um serviço simulado (stub), na porta 8080, que aguarda um intervalo configurável e devolve uma resposta JSON, e um agregador Spring Boot, na porta 8081, com três endpoints: `/aggregate/platform`, que faz uma chamada HTTP bloqueante na thread do pedido; `/aggregate/virtual`, que submete a chamada bloqueante a uma virtual thread [2]; e `/aggregate/async`, que usa `HttpClient.sendAsync()` e `CompletableFuture`.

O gerador k6 aumenta a carga até ao nível pretendido durante 10 s, mantém-na durante 30 s e reduz a carga durante 5 s. Foram executados os três modos com 10, 100 e 500 utilizadores virtuais (VUs). Recolheram-se throughput, latências p50/p95/p99 e taxa de pedidos falhados, métricas disponibilizadas pelo k6 [1]. Os resultados brutos estão guardados em JSON, com resumo CSV e gráficos.

2 Configuração e resultados do piloto

O ficheiro de configuração atual do stub define `stub.delay-ms=1000`; logo, o piloto usou um atraso artificial de 1.000 ms. O teste define timeout de 15 s. O registo de ambiente indica Windows 10, Temurin OpenJDK 25.0.5 (target do projeto: Java 21) e Maven 3.10.0. A documentação do benchmark indica que o gerador e os servidores foram executados na mesma máquina. O modelo do processador e a memória RAM ainda não estão registados.

Existe uma execução guardada por combinação de modo e carga. A Tabela 1 resume as métricas a 500 VUs. Todos os nove resumos registam 0% de pedidos falhados.

Table 1. Resultados do piloto a 500 VUs (uma execução por modo).

Modo	req/s	p50 (ms)	p95 (ms)	p99 (ms)
Platform	183,80	2.523,02	2.564,51	2.575,87
Virtual	183,68	2.511,24	2.642,35	2.707,13
Async	184,35	2.513,77	2.572,03	2.604,67

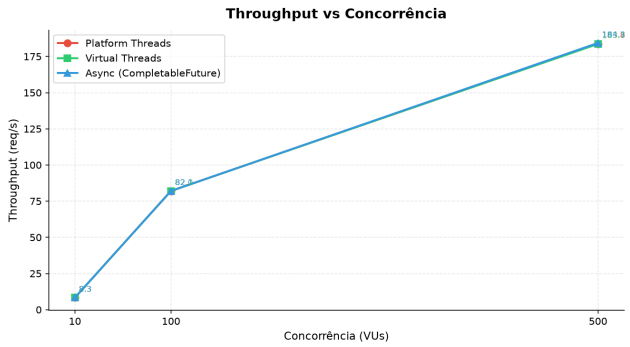


Figure 1. Throughput no piloto exploratório. Os três modos apresentam valores próximos.

3 Interpretação e limitações

Nos níveis de carga testados, os dados guardados não mostram uma vantagem clara de throughput para qualquer modo. A 10 VUs, a latência p50 situa-se entre 1.007 e 1.013 ms, consistente com o atraso configurado no stub. A 500 VUs, o throughput fica perto de 184 pedidos/s e o p99 entre 2,6 e 2,7 s. Como há apenas uma execução por condição, estes valores descrevem o piloto e não demonstram equivalência nem superioridade de um modelo.

Há ainda um detalhe a resolver no modo virtual: o controlador submete o trabalho a uma virtual thread e chama `Future.get()`. Assim, a thread do pedido no Tomcat fica bloqueada até a tarefa terminar; não é libertada durante a espera pelo stub. Esta implementação deve ser corrigida, ou a comparação deve ser interpretada tendo em conta essa limitação, antes das medições finais.

4 Próximos passos

Corrigir e validar o percurso de virtual threads; registar CPU e RAM; repetir a comparação base várias vezes, alternando ou aleatorizando a ordem dos modos e guardando os dados brutos. Depois de estabilizar a baseline, variar um fator de cada vez: atraso variável, 1/3/5 chamadas ao stub e trabalho de CPU. Medir também memória, analisar a dispersão dos resultados e completar o artigo com metodologia reproduzível e referências.

5 Utilização de ferramentas de IA

Ferramentas de IA generativa foram usadas para apoiar a redação e revisão deste relatório.

References

[1] Grafana Labs. 2026. Built-in metrics reference. https://grafana.com/docs/k6/latest/using-k6/metrics/reference/. Accessed 8 November 2026.

[2] OpenJDK. 2023. JEP 444: Virtual Threads. https://openjdk.org/jeps/444. Accessed 8 November 2026.